BYOP Project Report

Student Performance Predictor

---

1. Problem Statement

In schools and colleges, it is often difficult to know which students are at risk of failing until it is too late. By the time exam results come out, there is no time left to help them.

This project solves that problem using machine learning. It takes three simple inputs about a student — how much they study, how often they attend class, and what they scored previously — and predicts whether they will Pass or Fail. Teachers can use this tool to identify struggling students early and take action before exams.

2. Why This Problem Matters

- Early warning helps teachers give extra attention to weak students before it is too late.
- The three inputs used are already tracked by most schools, so no extra data collection is needed.
- A simple ML model can be more objective than manual guesswork by a teacher.
- This directly applies the classification concepts taught in the AI/ML course.

3. Approach and Solution

3.1 Dataset

A dataset of 300 student records was created using Python's NumPy library. Each record has three features and one label:

Feature	Description
study_hours	How many hours the student studies per day (1 to 9)
attendance	What percentage of classes the student attended (40 to 100)
prev_score	The student's score in the previous exam (20 to 100)
result (label)	Pass or Fail — this is what the model learns to predict

The Pass or Fail label is calculated using a weighted formula. Students who study more, attend more, and scored better previously are more likely to pass. A small amount of random noise is added to make the data realistic.

3.2 Models Used

Three machine learning models were trained and compared on the same data:

- Logistic Regression — a simple linear model, good as a baseline.
- Decision Tree — learns rules by splitting data at different points.
- Random Forest — combines many decision trees to give a more accurate result.

The model with the best accuracy on the test set is automatically selected as the best model.

3.3 Steps in the Code

- Step 1 — Create the dataset and save it as students.csv
- Step 2 — Draw 3 charts to understand the data (EDA)
- Step 3 — Scale the features and split into train/test sets (80/20)
- Step 4 — Train all 3 models and compare their accuracy
- Step 5 — Evaluate the best model, print report, save confusion matrix chart
- Step 6 — Save the model to disk using pickle
- Step 7 — Predict for new students and run the interactive prompt

4. Key Decisions Made

Decision	Reason
Used StandardScaler	Features are on different scales. Scaling makes comparison fair for all models.
Used 80/20 split	Standard split: 80% to train the model, 20% to test how well it works on unseen data.
Compared 3 models	Comparing models is an intermediate-level technique. It helps find the best one rather than guessing.
Used 5-fold cross-validation	CV gives a more reliable accuracy estimate than a single train/test split.
Saved model with pickle	So the trained model can be loaded and reused without retraining every time.
Single file (main.py)	Easier to submit, read, and run. All logic flows from top to bottom.

5. Results

The three models were compared and the best one was selected automatically. Typical results on the test set (60 students) are shown below:

Model	Test Accuracy	CV Score (5-fold)
Logistic Regression	~86%	~85%
Decision Tree	~88%	~87%
Random Forest (best)	~92%	~90%

Random Forest gave the best accuracy. Its feature importance chart showed that Previous Score had the biggest impact on the prediction, followed by Study Hours and Attendance.

6. Challenges Faced

Feature Scaling

Study hours (1–9), attendance (40–100), and prev_score (20–100) are on very different scales. Without scaling, the model gives more weight to larger numbers. StandardScaler was used to fix this — it transforms all features to have mean 0 and standard deviation 1.

Making Realistic Data

If the synthetic data is too clean, every model gets 100% accuracy and nothing is learned. A small noise value (0.05) was added to the label formula so the boundary between Pass and Fail is realistic and slightly imperfect — just like in real life.

Saving the Scaler

When predicting for a new student, the same scaler used during training must be applied. Forgetting to save and reload the scaler is a common mistake. In this project, the scaler is saved to model/scaler.pkl and reloaded in the predict function.

7. What I Learned

- How to build a complete ML pipeline from scratch: data → preprocessing → training → evaluation → prediction.
- Why it is important to scale features before training some ML models.
- How to compare multiple models and pick the best one using accuracy and cross-validation score.
- How to use a confusion matrix and classification report to understand model errors beyond just accuracy.
- How to save and load ML models using pickle so they can be reused.
- How to write clean, readable Python code organized in clear steps.

8. Future Improvements

- Use real school data instead of synthetic data for more meaningful results.
- Add more features such as homework marks, sleep hours, or internet access.
- Build a simple web interface using Streamlit so non-programmers can use it.
- Try more advanced models like Gradient Boosting or XGBoost.
- Add SHAP values to explain exactly why the model predicted Pass or Fail for each student.

9. Files Submitted

File	What It Contains
main.py	Complete project code — all 7 steps in one file
requirements.txt	List of Python libraries needed to run the project
README.md	Instructions on how to install and run the project
Project_Report.docx	This report
students.csv	Dataset (auto-created when main.py is run)
charts/ folder	All saved charts (auto-created when main.py is run)
model/ folder	Saved model and scaler files (auto-created)

End of Report

PRATYUSH PRASAD

25BCE11008